WEKALL · SQUAD POSEIDÓN

Prueba técnica — Desarrollador/a Semi Senior, Contact Center

Versión del 3 de noviembre de 2026

Contexto y reglas

La prueba tiene tres partes: unas 3 horas de trabajo propio y una sesión en vivo de 45 minutos. Mide lo que hace un desarrollador semi senior en el Contact Center de WeKall: manejar estados de agentes y colas en tiempo real, con backend en Java o Node, frontend en React y nociones de telefonía.

Parte	Qué es	Tiempo
A. Cuestionario	5 preguntas cortas sobre SIP, WebRTC y operación de Contact Center	30 min
B. Ejercicio práctico	Panel de agentes y colas en tiempo real	2 a 3 h, con plazo de 4 días para entregar
C. Sesión en vivo	Demo, revisión de código, un cambio pequeño y una pregunta de troubleshooting	45 min

Reglas

- **Backend:** Java 17+ con Spring Boot, o Node 20+ con NestJS o Express en TypeScript. Eliges uno.
- **Frontend:** React con TypeScript.
- **IA:** puedes usar Copilot, Claude, ChatGPT, Codex o la que prefieras. Indica en el README dónde y cómo la usaste. En la sesión en vivo vas a explicar cualquier parte del código.
- **Experiencia en telefonía:** no necesitas haber trabajado con Asterisk. Si no sabes una respuesta de la Parte A, explica cómo la averiguarías; eso también se evalúa.
- **Supuestos:** si algo del enunciado no es claro, decide, documenta el supuesto en el README y sigue.
- **Datos:** todos los datos son ficticios.

Parte A — Cuestionario de telefonía y Contact Center (30 min)

Responde por escrito, sin buscar en internet. Respuestas cortas y con tus palabras; valoramos el razonamiento más que la definición exacta.

1. **Flujo SIP.** Describe los mensajes principales de una llamada básica entre dos teléfonos, desde que se marca hasta que se cuelga.
2. **SIP y RTP.** ¿Qué transporta cada uno? Si un agente oye al cliente pero el cliente no lo oye a él, ¿qué revisarías primero?
3. **WebRTC.** ¿Qué necesita un webphone en el navegador para hacer llamadas? ¿Por qué suele fallar si la página no está en HTTPS?

4. **Métricas.** Explica con tus palabras qué son ASA, AHT y tasa de abandono.
5. **Diagnóstico.** Un agente aparece como *Disponible* en el tablero, pero no le entran llamadas mientras la cola tiene clientes esperando. Lista tres hipótesis y cómo verificarías cada una.

Parte B — Ejercicio práctico: panel de agentes y colas en tiempo real (2 a 3 h)

Construye un servicio que reciba eventos de llamadas, asigne cada llamada a un agente y muestre en un tablero React el estado de agentes y colas en tiempo real.

Modelo

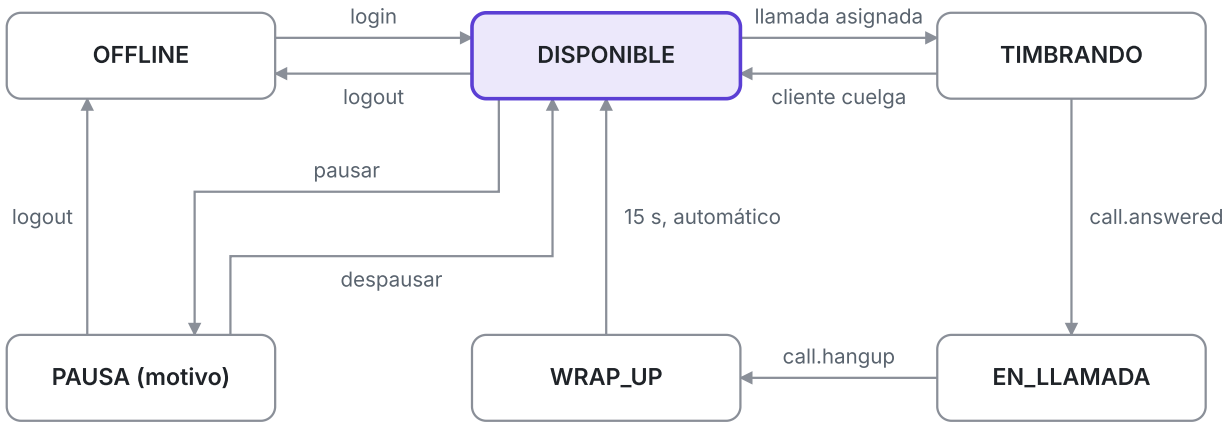
- **Agente:** id, nombre, extensión, colas a las que pertenece, estado actual y hora en que entró a ese estado.
- **Cola:** id, nombre y tiempo máximo de espera en segundos.
- **Llamada:** id, número de origen, cola, estado (`EN_COLA`, `TIMBRANDO`, `CONECTADA`, `FINALIZADA`, `ABANDONADA`) y marcas de tiempo de cada cambio.

Estados del agente

El agente solo puede moverse por las transiciones del diagrama. Cualquier otra transición se rechaza y queda registrada en el log.

Solo estas 10 transiciones son válidas; cualquier otra se rechaza

Solo un agente en DISPONIBLE puede recibir una llamada



`WRAP_UP` dura 15 segundos; después el agente vuelve solo a `DISPONIBLE`.

Eventos de entrada

Tu servicio recibe eventos por `POST /events`, uno por petición o en lote. Cada evento trae `eventId`, `ts` (ISO 8601) y `type`.

Evento	Campos	Qué debe pasar
`call.incoming`	`callId`, `queue`, `callerId`	La llamada entra a la cola. Si hay un agente disponible, se le asigna y pasa a `TIMBRANDO`.
`call.answered`	`callId`, `agentId`	El agente pasa a `EN_LLAMADA` y la llamada a `CONECTADA`.
`call.hangup`	`callId`	Si la llamada estaba en cola o timbrando, queda `ABANDONADA` y el agente que timbraba vuelve a `DISPONIBLE`. Si estaba conectada, queda `FINALIZADA` y el agente pasa a `WRAP_UP`.

Ejemplo de una secuencia:

```
{"eventId":"e-001","ts":"2026-11-10T14:00:00Z","type":"call.incoming","callId":"c-001","queue":"soporte","c
allerId":"+573001112233"}
{"eventId":"e-002","ts":"2026-11-10T14:00:06Z","type":"call.answered","callId":"c-001","agentId":"a-1001"}
{"eventId":"e-002","ts":"2026-11-10T14:00:06Z","type":"call.answered","callId":"c-001","agentId":"a-1001"}
{"eventId":"e-003","ts":"2026-11-10T14:04:31Z","type":"call.hangup","callId":"c-001"}
```

El tercer evento es un duplicado a propósito.

Requisitos del backend

1. **Asignación:** cuando entra una llamada o un agente queda disponible, asigna la llamada más antigua de la cola al agente `DISPONIBLE` de esa cola que lleve más tiempo libre.
2. **Una llamada, un agente:** una llamada no puede asignarse a dos agentes y un agente no puede recibir dos llamadas. Cuéntanos en el README cómo lo evitas.
3. **Duplicados:** un evento con un `eventId` ya procesado no cambia nada. Un evento que no aplica al estado actual, como un `call.answered` de una llamada ya colgada, se registra y se ignora sin romper el servicio.
4. **API REST:** login, logout, pausar, despausar, listar agentes y listar colas. Incluye ejemplos con `curl` en el README.
5. **Tiempo real:** publica los cambios de agentes y colas al frontend por WebSocket o SSE.
6. **Métricas por cola:** llamadas en espera, atendidas, abandonadas y la espera más larga en este momento.
7. **Persistencia:** guardar todo en memoria está bien.
8. **Script de prueba:** un script que envíe la secuencia de ejemplo, o una tuya, para ver el tablero en movimiento.
9. **Pruebas:** pruebas unitarias de la máquina de estados y de la asignación.

Requisitos del frontend

1. **Tablero de agentes:** estado con color por estado y el tiempo en el estado actual corriendo en vivo.
2. **Panel por cola:** las métricas del punto 6.
3. **Reconexión:** si se cae la conexión en tiempo real, el tablero lo muestra y reconecta solo.

Extras opcionales

Suman puntos, pero no reemplazan nada de lo obligatorio.

- Evento `call.noanswer`: el agente que no contesta pasa a `PAUSA` con motivo “No contesta” y la llamada vuelve al inicio de la cola.
- Tiempo máximo de espera por cola: al vencerse, la llamada sale como `ABANDONADA`.
- ASA y nivel de servicio con umbral de 20 s en el panel de colas.
- Botones de supervisor en el tablero para pausar y despausar agentes.
- Resincronizar el tablero después de reconectar, sin recargar la página.
- PostgreSQL y `docker compose` para levantar todo con un comando.
- Conectar el servicio a un Asterisk real por AMI o ARI, o un webphone mínimo con JsSIP o SIP.js.

Parte C — Sesión en vivo (45 min)

Es una conversación técnica sobre tu entrega, compartiendo pantalla.

Bloque	Tiempo	Qué pasa
Demo	10 min	Muestras el tablero con el script de prueba corriendo.
Revisión de código	15 min	Recorremos tus decisiones y las partes hechas con IA.
Cambio en vivo	15 min	Implementas un cambio pequeño que te damos en el momento. Puedes usar IA.
Troubleshooting	5 min	Te planteamos una falla de Contact Center y nos cuentas cómo la investigarías.

Entrega

Tienes 4 días calendario desde que recibes la prueba. Si necesitas más tiempo, avísalo antes de que se venza el plazo; no penaliza.

1. **Repositorio Git** en GitHub, GitLab o Bitbucket, público o con acceso para quien te envió la prueba. Haz commits pequeños: revisamos el historial.
2. **Respuestas de la Parte A** en un archivo `PARTE_A.md` dentro del repositorio.
3. **README** con:
   - cómo levantar el proyecto;
   - cómo evitas que una llamada se asigne dos veces;
   - supuestos que tomaste;
   - qué quedó pendiente y qué harías con más tiempo;
   - dónde y cómo usaste IA.